

Mean Chat

Comprehensive documentation for the Mean Chat project covering repository layout, data models, API design, Angular architecture, and client-server integration details.

Repository Organisation & Git Workflow

- **Top-level layout**
 - `server/` – Node.js + Express backend, Socket.IO gateway, MongoDB data access helpers, upload utilities, and REST routes under `routes/`.
 - `client/chat/` – Angular workspace for the web client (components, services, models, routing configuration, and Cypress e2e specs under `cypress/`).
 - Archive folders (`server.zip` , `client.zip`) are snapshots and not part of the active code path.
- **Suggested Git usage**
 - Keep `main` (or `master`) stable; branch per feature (`feature/api-messages` , `feature/ui-admin`) to isolate changes.
 - Commit on logical checkpoints (e.g., “Add channel ban API” + “Consume ban API in group-admin component”), referencing both server and client pieces when they belong together.
 - Run lint/tests (Angular unit/Cypress, server mocha) before merging; use pull requests to review how REST or socket changes impact the Angular client.
 - Global dependencies (`node_modules/`) are ignored via `.gitignore` and must never be committed.

Data Structures

Server Entities (`server/lib/db-mongo.js`)

Entity	Storage	Key Fields
User	MongoDB users collection	<code>id</code> , <code>username</code> , <code>usernameLower</code> , <code>email</code> , <code>emailLower</code> , <code>password</code> (PBKDF2 hashed), <code>roles: string[]</code> , <code>groups: string[]</code> , <code>avatarKey</code> , <code>avatarUrl</code> , <code>createdAt</code>
Group	MongoDB groups collection	<code>id</code> , <code>name</code> , <code>createdBy</code> , <code>admins: string[]</code> , <code>members: string[]</code> , <code>createdAt</code> (epoch ms), <code>avatarKey</code> , <code>avatarUrl</code>
Channel	MongoDB channels collection	<code>id</code> , <code>groupId</code> , <code>name</code> , <code>createdBy</code> , <code>createdAt</code> (epoch ms), <code>bannedUserIds: string[]</code>
Message	MongoDB messages collection	<code>id</code> , <code>groupId</code> , <code>channelId</code> , <code>userId</code> , <code>username</code> , <code>content</code> , <code>avatarKey</code> , <code>imageKey</code> , <code>imageContentType</code> , <code>imageFilename</code> , <code>timestamp</code>

The database module caches the active `MongoClient` , `database` , and `initPromise` at module scope so subsequent requests reuse the connection. ID generation is handled by `genId(prefix)` .

Client Models (`client/chat/src/app/models/`)

Interface	Source File	Fields
-----------	-------------	--------

User	<code>user.model.ts</code>	<code>id, username, email, roles, groups, optional avatarUrl</code>
Group	<code>group.model.ts</code>	<code>id, name, createdBy, admins, members, createdAt: Date, optional avatarUrl</code>
Channel	<code>channel.model.ts</code>	<code>id, name, groupId, createdBy, createdAt: Date</code>
Additional view models	Component-local state uses the above interfaces plus lightweight types (e.g., message DTOs) defined inline where needed.	

Responsibilities: Client vs. Server

- **Server (`server/`)**

- Owns persistence (MongoDB via `lib/db-mongo.js`) and enforces business rules for authentication, authorisation, memberships, bans, and cascading deletes.
- Exposes a REST API under `/api` returning JSON payloads for auth, users, groups, channels, messages, admin import/export, and media streaming.
- Manages file uploads (avatars, group avatars, message images) via `multer`, storing files on disk (under `uploads/`) or via `media.saveBuffer` abstractions.
- Provides Socket.IO real-time events (`lib/socket.js`) for new chat messages and call signalling.
- Controls environment-driven behaviour through `MEAN_CHAT_*` variables (Mongo URI/DB, upload root, PBKDF2 iterations, media secret).

- **Client (`client/chat/`)**

- Presents the UI using Angular standalone components, routing, and services.
- Maintains session state in `AuthService` with persistence via `localStorage` and an HTTP interceptor that adds `x-user-id` to REST calls.
- Fetches and mutates data via `HttpClient` using typed services (`UserService` , `GroupService` , `MessagesService` , etc.).
- Listens for Socket.IO updates to inject live messages into component state.
- Handles file uploads (avatars, message images) through Angular forms and posts to corresponding REST endpoints.

REST API Reference (Base URL: `/api`)

Auth

Method	Path	Request Body	Response	Purpose
POST	<code>/auth/login</code>	<code>{ username, password }</code>	<code>{ success, user }</code>	Authenticate default super admin or newly registered users.
POST	<code>/auth/register</code>	<code>{ username, email, password }</code>	<code>{ success, user }</code>	Register a new user; validation ensures unique username/email.

Users (requires `x-user-id` header)

Method	Path	Request Body / Params	Response	Purpose
POST	/users	Super admin only; { username, email, password, roles?, groups? }	201 { success, user }	Create a user with optional roles/groups.
GET	/users	–	User[]	List all users (roles filtered on client).
GET	/users/:id	URL param id	User	Fetch single user.
PUT	/users/:id	Self or super; { username?, email?, roles? }	Updated User	Update core profile fields.
POST	/users/:id/avatar	Multipart avatar image	{ success, user }	Upload & process avatar (sharp-resized JPEG).
PATCH	/users/:id/roles	Super only; { roles: string[] }	Updated User	Replace role set.
POST	/users/:id/promote	Super only	Updated User	Add group-admin role.
DELETE	/users/:id	Super only	{ success: true }	Delete user, clean group memberships/admins.

Groups (requires `x-user-id`)

Method	Path	Request	Response	Purpose
GET	/groups	–	Group[] (scoped: super sees all, others see theirs)	List groups for current user.
GET	/groups/user/:userId	URL param	Group[]	List groups for any user (super gets all, others filtered).
POST	/groups	Group admins or super; { name }	{ success, group }	Create group; creator becomes admin/member.
POST	/groups/:groupId/avatar	Multipart avatar	{ success,	Upload group avatar; saves via

			group }	media.saveBuffer.
DELETE	/groups/:groupId	Owner or super	{ success: true }	Delete group and associated channels/messages, cleanup media.
POST	/groups/:groupId/members	Admin or super; { userId }	Group	Add member.
DELETE	/groups/:groupId/members/:userId	Admin or super	Group	Remove member.
POST	/groups/:groupId/channels/:channelId/ban	Admin or super; { userId }	{ success, channel }	Add user to channel ban list.

Channels (requires x-user-id)

Method	Path	Request	Response	Purpose
GET	/channels/group/:groupId	URL param	Channel[]	List channels within group.
POST	/channels	Admin or super; { name, groupId }	Channel	Create channel.
DELETE	/channels/:channelId	Admin or super	{ success: true }	Delete channel (removes messages & bans).

Messages (requires x-user-id)

Method	Path	Request	Response	Purpose
GET	/messages	Query: channelId (required), optional groupId, limit	{ success, messages }	Fetch latest messages (default limit 50).
POST	/messages	{ groupId, channelId, content }	{ success, message }	Persist text message, broadcast via Socket.IO.
POST	/messages/upload	Multipart image, body { groupId, channelId, content? }	{ success, message }	Upload image + optional text; stored via media.saveBuffer.

Admin & Media

Method	Path	Request	Response	Purpose
GET	/admin/export	Super only	{ success, data }	Export snapshot of users/groups/channels/messages.

POST	/admin/import	Super only, { data }	{ success: true }	Replace dataset with provided snapshot.
GET	/media/:token	Signed token	Binary stream	Serve protected media (local file or encrypted buffer).

Real-Time / Socket.IO

- `lib/socket.js` namespaces messages to `channel:<channelId>` rooms and handles call signalling via `call:<groupId>` rooms. `emitNewMessage` decorates messages with media URLs before broadcasting.

Socket Messaging

The Angular client maintains a Socket.IO connection to the server for two-way updates in addition to the REST API. The server authenticates sockets using the `userId` supplied in the connection handshake (`handshake.auth.userId` , `query.userId` , or `x-user-id` header) and decorates `socket.data.user` with the resolved profile. All socket handlers provide optional acknowledgements; when supplied, the server replies with `{ success: boolean, ... }` .

Client → Server Events

- `joinChannel { channelId, groupId? }` — validates membership/ban status, joins the `channel:<id>` room, returns the latest messages (decorated with media URLs), the current presence list, and any active call metadata. Triggers `channel:user-joined` for other room members.
- `leaveChannel { channelId }` — removes the socket from the channel room and returns the updated presence list. Emits `channel:user-left` to remaining members.
- `chat:message { channelId, groupId?, content }` — persists a text message through the same helper as the REST endpoint, emits `chat:message` to the room, and acknowledges with the decorated message payload. Requires having joined the channel first.
- `call:join { channelId, groupId? }` — subscribes to the `call:<id>` room, manages the active call state (`activeCalls` map), and starts a call if none exists. Returns the list of other participants. May also emit `call:started` to the channel room and record a system message.
- `call:leave { channelId }` — leaves the call room; if the last participant departs the call ends, `call:ended` is broadcast, and a system message is stored.
- `call:signal { channelId, type, data }` — forwards WebRTC signalling payloads to other call participants in the room. The server ignores signals if the sender has not joined the call room.

Server → Client Broadcasts

- `chat:message` — emitted to `channel:<id>` rooms whenever a message is created (via sockets or REST). Payload includes media URLs derived from `media.applyMessageMedia` .
- `channel:user-joined` / `channel:user-left` — presence notifications containing `{ channelId, userId, username, avatarUrl }` .
- `call:started` / `call:ended` — announced to channel listeners when a call begins or ends, including timestamps and the initiating/ending user.
- `call:user-joined` / `call:user-left` — keep call participants informed about who is in the conference.
- `call:signal` — relays WebRTC offer/answer/ICE data between peers. Payload includes `{ channelId, from, type, data }` .

Presence state is tracked per channel via `channelMembers` ; the server cleans up maps on disconnect. Active call metadata (`participants` , `startedBy` , `startedAt`) is stored in `activeCalls` to support rejoining clients and to ensure REST-created system messages reflect call lifecycle events.

Angular Architecture

Angular is configured via `client/chat/src/app/app.config.ts` where the router, HTTP client (with the `authInterceptor`), and animations support are provided. Routes live in `client/chat/src/app/app-routing.module.ts` and are guarded by `AuthGuard` so only authenticated users (optionally matching required roles) can access protected views. The app bootstraps a standalone `AppComponent` shell that renders the active route alongside global overlays for toasts and confirmation dialogs.

Components

Component	Source	Responsibilities
AppComponent	<code>client/chat/src/app/app.component.ts</code>	Root shell hosting RouterOutlet, ToastsComponent and ConfirmComponent so overlays persist across route changes.
LoginComponent	<code>client/chat/src/app/components/login/login.component.ts</code>	Handles login/registration forms using reactive forms; dispatches auth requests, and toggles between modes.
DashboardComponent	<code>client/chat/src/app/components/dashboard/dashboard.component.ts</code>	Authenticated landing view: loads groups/channel manages profile editing and avatar upload and routes into chat/admin areas.
ChatComponent	<code>client/chat/src/app/components/chat/chat.component.ts</code>	Real-time chat UI driven by Angular signals; joins sockets, renders message timeline, handles

		uploads, and surfaces call entry points.
CallComponent	client/chat/src/app/components/call/call.component.ts	WebRTC meeting surface with participant tiles, screen sharing, speaking indicators, and signalling subscriptions
AdminComponent	client/chat/src/app/components/admin/admin.component.ts	Super-admin dashboard for user provisioning, updates, and optional group assignment.
GroupAdminComponent	client/chat/src/app/components/group-admin/group-admin.component.ts	Group admin workspace for channel creation, membership management, and avatar uploads.
ToastsComponent	client/chat/src/app/components/toasts/toasts.component.ts	Renders toast notifications streamed from NotifyService
ConfirmComponent	client/chat/src/app/components/confirm/confirm.component.ts	Presents confirmation dialogs driven by ConfirmService handling accept/cancel actions.

Supporting UI helpers include `client/chat/src/app/directives/media-stream.directive.ts`, which binds `MediaStream` instances to video elements for the call experience.

Services

Service	Source	Role
AuthService	client/chat/src/app/services/auth.service.ts	Manages authentication state via BehaviorSubject, persists the

		current user in <code>StorageService</code> , exposes login/register/logout APIs, and provides role helpers for guards/UI.
UserService	<code>client/chat/src/app/services/user.service.ts</code>	Wraps REST calls for user CRUD, promotion, and avatar operations.
GroupService	<code>client/chat/src/app/services/group.service.ts</code>	Provides group/channel membership APIs, including avatar uploads and listings for specific users.
SocketService	<code>client/chat/src/app/services/socket.service.ts</code>	Lazily establishes Socket.IO connections, exposes typed observables for server broadcasts, and offers ack-based helpers (<code>joinChannel</code> , <code>leaveChannel</code> , <code>emitWithAck</code>) for two-way messaging.
NotifyService	<code>client/chat/src/app/services/notify.service.ts</code>	Publishes toast notifications through a <code>BehaviorSubject</code> consumed by <code>ToastsComponent</code> .
ConfirmService	<code>client/chat/src/app/services/confirm.service.ts</code>	Manages confirmation dialog state, exposing a promise-based API to components.
StorageService	<code>client/chat/src/app/services/storage.service.ts</code>	Encapsulates <code>localStorage</code> read/write/clear helpers.

Infrastructure pieces:

- `client/chat/src/app/interceptors/auth.interceptor.ts` injects the `x-user-id` header on every HTTP request if a user is stored locally.
- `client/chat/src/app/guards/auth.guard.ts` enforces authentication and optional role constraints per route.

Models & Shared Types

Model/Type	Source	Description
User	<code>client/chat/src/app/models/user.model.ts</code>	Client representation of a user (id, username, email, roles, groups, optional avatar URL).
Group	<code>client/chat/src/app/models/group.model.ts</code>	Group metadata including admins, members, creation date, and optional avatar URL.

Channel	<code>client/chat/src/app/models/channel.model.ts</code>	Channel identity tied to a specific group with creator and timestamp.
Socket payloads	<code>client/chat/src/app/services/socket.service.ts</code> (<code>ServerMessage</code> , <code>ChannelPresenceEvent</code> , <code>CallSessionEvent</code> , etc.)	Type definitions for socket acknowledgements and broadcasts that keep chat and call components strongly typed.
View models	<code>ChatMessage</code> (in <code>chat.component.ts</code>), <code>RemoteParticipant</code> (in <code>call.component.ts</code>)	Component-scoped interfaces that extend server data with presentation-specific fields (e.g., computed timestamps, media streams).

This layered structure keeps components declarative, pushes side effects to services, and shares strongly typed models between the view layer and transport code.

Client–Server Interaction Details

Scenario	Server Side Changes	Client Side Updates
Authentication	<code>/api/auth/login</code> checks credentials via <code>findUserByCredentials</code> (Mongo lookup + PBKDF2 verify). Successful logins return the full user (minus password).	<code>AuthService</code> stores user in <code>localStorage</code> , <code>AuthGuard</code> allows navigation, and the interceptor includes <code>x-user-id</code> on future requests. Login view (<code>LoginComponent</code>) reacts by routing to <code>/dashboard</code> .
Group & Channel Management	Routes in <code>server/routes/groups.js</code> / <code>channels.js</code> mutate Mongo documents via <code>db-mongo.js</code> helpers (<code>createGroup</code> , <code>addGroupMember</code> , <code>createChannel</code> , <code>updateChannelBans</code>). Avatar uploads call <code>media.saveBuffer</code> , updating <code>avatarKey</code> and deleting previous files through <code>uploads.js</code> .	<code>AdminComponent</code> and <code>GroupAdminComponent</code> call corresponding services; observables update Angular component state arrays. The templates re-render lists/tables, and <code>NotifyService</code> toasts provide feedback.
Messaging	<code>server/routes/messages.js</code> validates membership/bans, persists via <code>createMessage</code> , and fires <code>emitNewMessage</code> . File uploads write to disk and store <code>imageKey</code> . Shared module-level <code>getDb()</code> caches ensure minimal connection churn.	<code>ChatComponent</code> posts via <code>MessagesService</code> . On success, the component prepends the returned message. Additionally, the Socket.IO client listens for <code>chat:message</code> events (decorated with media URLs by <code>applyMessageMedia</code>) to keep the timeline in sync across users.

Calls & Screen Sharing	Socket namespaces (<code>callRoom</code>) coordinate signalling; no persistent data is stored, but in-memory server references track the active Socket.IO server instance.	<code>CallComponent</code> uses <code>SocketService</code> to negotiate WebRTC connections. UI updates (participant tiles, screen-share indicators) rely on Angular signals/observables triggered by socket events.
Admin Import/Export	<code>/api/admin/export</code> aggregates collections via <code>exportData</code> . <code>/api/admin/import</code> replaces documents, hashing passwords when necessary and rebuilding indexes.	<code>AdminComponent</code> exposes buttons to download or upload datasets; file import uses Angular <code>HttpClient</code> to post JSON, followed by refresh of user/group data to reflect new state.
Media Delivery	<code>/api/media/:token</code> decodes signed tokens, reads from disk (<code>resolveFilePathFromUrl</code>) or encrypted storage, and streams the file.	Components display media URLs generated by <code>media.apply*</code> helpers (e.g., avatars, message images). When data updates (new avatar), Angular bindings update the <code></code> sources automatically.

Environment variables (`MEAN_CHAT_MONGO_URI` , `MEAN_CHAT_DB_PROVIDER` , `MEAN_CHAT_UPLOAD_ROOT` , `MEAN_CHAT_MEDIA_SECRET` , etc.) act as global configuration on the server. Changing them affects DB connections, upload paths, and media token secrets without altering code; Angular configuration reads API endpoints from service classes currently targeting `http://localhost:3000/api` .

Real-Time Collaboration Highlights

Video Chatting & Screen Sharing

- `client/chat/src/app/components/call/call.component.ts` keeps the full WebRTC pipeline on the client, using Angular `signal` / `computed` state to coordinate call lifecycles, audio-level analysis, and the `toggleScreenShare()` bridge to `navigator.mediaDevices.getDisplayMedia` .
- `client/chat/src/app/components/call/call.component.html` renders host badges, mute indicators, a dedicated screen-share tile, and hot-swappable local/remote `<video>` elements that receive live `MediaStream` objects via `client/chat/src/app/directives/media-stream.directive.ts` .
- `server/lib/socket.js` enforces channel membership with `ensureChannelAccess` , maintains per-channel call registries, and emits `call:*` events that `SocketService` fans out to the UI so participants join/leave instantly without polling.

Angular Signals & Animations

- `client/chat/src/app/components/chat/chat.component.ts` and `client/chat/src/app/components/call/call.component.ts` lean on Angular signals for view-model state (`messages` , `callActive` , `speakingParticipants`), computed guards (e.g., `disableSend` , `isScreenSharing`), and mutation helpers that keep the templates reactive without manual change detection.
- New chat messages animate into view through the `trigger('messageFade', ...)` definition in `client/chat/src/app/components/chat/chat.component.ts` , while the call surface uses structural directives bound to signals to cross-fade between lobby, live tiles, and control palettes.

- Shared services such as `client/chat/src/app/services/socket.service.ts` pair RxJS streams with the signal-based components, giving a clean boundary between transport events and declarative UI reactions.

Backend Storage: Efficiency vs Security

- `server/lib/db-mongo.js` reuses a singleton `MongoClient` with configurable pooling (`MEAN_CHAT_MONGO_MAX_POOL`), primes compound indexes for high-volume queries, and stores credentials as PBKDF2 hashes validated with `crypto.timingSafeEqual` .
- Encrypted media at rest lives under `server/lib/media.js` , which derives AES-256-GCM keys from `MEAN_CHAT_MEDIA_SECRET` , rotates per-install secrets when missing, and issues signed download tokens so the UI can stream avatars and attachments without exposing raw file paths.
- Request paths such as `server/routes/messages.js` and the socket gateway (`server/lib/socket.js`) double-check group membership, ban lists, and file constraints, balancing fast channel lookups against strict access control before data ever hits MongoDB.

Development Notes

1. Setup

- Server: `cd server && npm install && npm start` (requires MongoDB instance; defaults to `mongodb://127.0.0.1:27017` with DB `mean_chat`).
- Client: `cd client/chat && npm install && npm start` (Angular dev server on `http://localhost:4200`).
- Real-time features rely on running both services.

2. Testing

- Server: `npm test` inside `server/` runs Mocha suites (`__tests__` /).
- Client: `npm test` for Angular unit tests, `npm run cypress:run` for headless e2e (Firefox) using shared constants.

3. Repository hygiene

- Ensure `node_modules/` (both server and client) remain excluded from Git history; rely on package-lock files for deterministic installs.
- Environment-specific files (e.g., `.env`) should be ignored or templated if introduced.

This README should accompany the root of the repository and provide the necessary context for future development, review, and deployment activities.